

TASK 1

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 1000
```

```
/*
```

```
    TASK 1:
```

```
    Single Quote Parser
```

```
    Requirements:
```

- Preserve literal content
- Ignore variable expansion
- Store quoted strings
- Validate parsing results
- Test edge cases

```
*/
```

```
int parse_single_quotes(const char *input, char *output)
```

```
{
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    int inside_quotes = 0;
```

```
    while (input[i] != '\0')
```

```
    {
```

```

        if (input[i] == '\")
        {
            inside_quotes = !inside_quotes;
        }
        else
        {
            output[j++] = input[i];
        }

        i++;
    }

    output[j] = '\0';

    if (inside_quotes)
        return 0; // Invalid / unmatched quote

    return 1; // Valid
}

void test_case(const char *input)
{
    char output[MAX];

    printf("\nInput : %s\n", input);

    if (parse_single_quotes(input, output))
    {
        printf("Valid : Yes\n");
    }
}

```

```

        printf("Output: %s\n", output);
    }
    else
    {
        printf("Valid : No\n");
        printf("Output: Unmatched single quote\n");
    }
}

int main()
{
    printf("=====\n");
    printf("    SINGLE QUOTE PARSER\n");
    printf("=====\n");

    test_case("'Hello World'");

    test_case("'Hello $USER'");

    test_case("'This is a literal string'");

    test_case("'Hello 'World''");

    test_case("'Single quoted text with spaces'");

    test_case("'Hello'");

    test_case('');

```

```

printf("\n=====\\n");

printf("Parsing tests completed.\\n");

printf("=====\\n");

return 0;
}

```

```

nikith@NIKITH:~$ mkdir -p ~/ossp
cd ~/ossp
nikith@NIKITH:~/ossp$ nano single_quotes.c
nikith@NIKITH:~/ossp$ nano single_quotes.c
nikith@NIKITH:~/ossp$ gcc single_quotes.c -o single_quotes
nikith@NIKITH:~/ossp$ ./single_quotes
=====
          SINGLE QUOTE PARSER
=====

Input : 'Hello World'
Valid : Yes
Output: Hello World

Input : 'Hello $USER'
Valid : Yes
Output: Hello $USER

Input : 'This is a literal string'
Valid : Yes
Output: This is a literal string

Input : 'Hello 'World''
Valid : Yes
Output: Hello World

Input : 'Single quoted text with spaces'
Valid : Yes
Output: Single quoted text with spaces

Input : 'Hello
Valid : No
Output: Unmatched single quote

Input : ''
Valid : Yes
Output:

=====
Parsing tests completed.
=====
nikith@NIKITH:~/ossp$ |

```

TASK 2

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX 1000
```

```
/*
```

TASK 2:

Double Quote Parser

Requirements:

- Preserve spaces
- Allow variable expansion
- Parse nested tokens
- Validate outputs
- Test quoted commands

```
*/
```

```
void expand_variables(const char *input, char *output)
```

```
{
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    while (input[i] != '\0')
```

```
    {
```

```
        if (input[i] == '$')
```

```
        {
```

```
            i++;
```

```
            char variable[100];
```

```

int k = 0;

/* ${VARIABLE} */
if (input[i] == '{')
{
    i++;

    while (input[i] != '\0' &&
           input[i] != '}' &&
           k < 99)
    {
        variable[k++] = input[i++];
    }

    if (input[i] == '}')
        i++;
}

/* $VARIABLE */
else
{
    while (input[i] != '\0' &&
           (isalnum((unsigned char)input[i]) ||
            input[i] == '_') &&
           k < 99)
    {
        variable[k++] = input[i++];
    }
}

```

```
variable[k] = '\0';
```

```
const char *value = getenv(variable);
```

```
if (value != NULL)
```

```
{
```

```
    strcpy(output + j, value);
```

```
    j += strlen(value);
```

```
}
```

```
else
```

```
{
```

```
    /* Keep unknown variables unchanged */
```

```
    output[j++] = '$';
```

```
    strcpy(output + j, variable);
```

```
    j += strlen(variable);
```

```
}
```

```
}
```

```
else
```

```
{
```

```
    output[j++] = input[i++];
```

```
}
```

```
}
```

```
output[j] = '\0';
```

```
}
```

```
int parse_double_quotes(const char *input, char *output)
```

```
{
```

```
int i = 0;
int j = 0;
int inside_quotes = 0;
```

```
while (input[i] != '\0')
{
    if (input[i] == '"')
    {
        inside_quotes = !inside_quotes;
    }
    else
    {
        output[j++] = input[i];
    }

    i++;
}
```

```
output[j] = '\0';
```

```
if (inside_quotes)
    return 0;
```

```
return 1;
}
```

```
void test_case(const char *input)
{
    char parsed[MAX];
```

```

char expanded[MAX];

printf("\nInput : %s\n", input);

if (!parse_double_quotes(input, parsed))
{
    printf("Valid : No\n");
    printf("Output: Unmatched double quote\n");
    return;
}

expand_variables(parsed, expanded);

printf("Valid : Yes\n");
printf("Output: %s\n", expanded);
}

int main()
{
    printf("=====\n");
    printf("    DOUBLE QUOTE PARSER\n");
    printf("=====\n");

    test_case("\"Hello World\"");

    test_case("\"Hello $USER\"");

    test_case("\"Home directory: $HOME\"");

```

```
test_case("\Hello $USER, welcome to Ubuntu\");

test_case("\This contains spaces\");

test_case("\Value: ${HOME}\");

test_case("\Hello");

test_case("\");

printf("\n=====\\n");
printf("Parsing tests completed.\\n");
printf("=====\\n");

return 0;
}
```

```
nikith@NIKITH:~$ nano double_quotes.c
nikith@NIKITH:~$ gcc double_quotes.c -o double_quotes
nikith@NIKITH:~$ ./double_quotes
=====
DOUBLE QUOTE PARSER
=====

Input : "Hello World"
Valid : Yes
Output: Hello World

Input : "Hello $USER"
Valid : Yes
Output: Hello nikith

Input : "Home directory: $HOME"
Valid : Yes
Output: Home directory: /home/nikith

Input : "Hello $USER, welcome to Ubuntu"
Valid : Yes
Output: Hello nikith, welcome to Ubuntu

Input : "This contains spaces"
Valid : Yes
Output: This contains spaces

Input : "Value: ${HOME}"
Valid : Yes
Output: Value: /home/nikith

Input : "Hello
Valid : No
Output: Unmatched double quote

Input : ""
Valid : Yes
Output:

=====
Parsing tests completed.
=====
nikith@NIKITH:~$ |
```